

Difference between autograd.grad and autograd.backward?

Asked 3 years, 5 months ago Modified 11 months ago Viewed 12k times



18



Suppose I have my custom loss function and I want to fit the solution of some differential equation with help of my neural network. So in each forward pass, I am calculating the output of my neural net and then calculating the loss by taking the MSE with the expected equation to which I want to fit my perceptron.

Now my doubt is: should I use `grad(loss)` or should I do `loss.backward()` for backpropagation to calculate and update my gradients?

I understand that while using `loss.backward()` I have to wrap my tensors with `Variable` and have to set the `requires_grad = True` for the variables w.r.t which I want to take the gradient of my loss.

So my questions are :

- Does `grad(loss)` also requires any such explicit parameter to identify the variables for gradient computation?
- How does it actually compute the gradients?
- Which approach is better?
- what is the main difference between the two in a practical scenario.

It would be better if you could explain the practical implications of both approaches because whenever I try to find it online I am just bombarded with a lot of stuff that isn't much relevant to my project.

`pytorch` `gradient` `gradient-descent` `backpropagation` `autograd` [Edit tags](#)

[Share](#) [Edit](#) [Follow](#) [Close](#) [Flag](#)

edited Jun 4, 2023 at 0:25



[kmario23](#)

61.4k ● 16 ● 171 ● 158

asked Sep 12, 2021 at 5:08



[unstableEquilibrium](#)

337 ● 1 ● 2 ● 9

3 Answers

Sorted by: [Reset to default](#) Date modified (newest first)



24



TLDR; Both are different interfaces for gradient computation: `torch.autograd.grad` is non-mutable while `torch.autograd.backward` is.

Descriptions

The `torch.autograd` module is the automatic differentiation package for PyTorch. As described in the documentation it only requires minimal change to the code base to be used:

you only need to declare `Tensor`s for which gradients should be computed with the `requires_grad=True` keyword.

The two main functions `torch.autograd` provides for gradient computation are `torch.autograd.backward` and `torch.autograd.grad`:

	torch.autograd.backward (source)	torch.autograd.grad (source)
Description	Computes the sum of gradients of given tensors with respect to graph leaves .	Computes and returns the sum of gradients of outputs with respect to the inputs .
Header	<pre>torch.autograd.backward(tensors, grad_tensors=None, retain_graph=None, create_graph=False, grad_variables=None, inputs=None)</pre>	<pre>torch.autograd.grad(outputs, inputs, grad_outputs=None, retain_graph=None, create_graph=False, only_inputs=True, allow_unused=False)</pre>
Parameters	- <code>tensors</code> – Tensors of which the derivative will be computed. - <code>grad_tensors</code> – The "vector" in the Jacobian-vector product, usually gradients w.r.t. each element of corresponding tensors. - <code>retain_graph</code> – If <code>False</code>, the graph used to compute the grad will be freed. [...] - <code>inputs</code> – Inputs w.r.t. which the gradient be will be accumulated into <code>.grad</code>. All other Tensors will be ignored. If not provided, the gradient is accumulated into all the leaf Tensors that were used [...].	- <code>outputs</code> – outputs of the differentiated function. - <code>inputs</code> – Inputs w.r.t. which the gradient will be returned (and not accumulated into <code>.grad</code>). - <code>grad_outputs</code> – The "vector" in the Jacobian-vector product, usually gradients w.r.t. each element of corresponding tensors. - <code>retain_graph</code> – If <code>False</code>, the graph used to compute the grad will be freed. [...].

Usage examples

In terms of high-level usage, you can look at `torch.autograd.grad` as a non-mutable function. As mentioned in the documentation table above, it will not accumulate the gradients on the `grad` attribute but instead return the computed partial derivatives. In contrast `torch.autograd.backward` will be able to mutate the tensors by updating the `grad` attribute of leaf nodes, the function won't return any value. In other words, the latter is more suitable when computing gradients for a large number of parameters.

In the following, we will take two inputs (`x1` and `x2`), calculate a tensor `y` with them, and then compute the partial derivatives of the result w.r.t both inputs, i.e. `dL/dx1` and `dL/dx2`:

```
>>> x1 = torch.rand(1, requires_grad=True)  
>>> x2 = torch.rand(1, requires_grad=True)  
>>> x1, x2  
(tensor(0.3939, grad_fn=<UnbindBackward>),  
 tensor(0.7965, grad_fn=<UnbindBackward>))
```

Inference:

```
>>> y = x1**2 + 5*x2
>>> y
tensor(4.1377, grad_fn=<AddBackward0>)
```

Since `y` was computed using tensor(s) requiring gradients (i.e. with `requires_grad=True`) - *outside of a `torch.no_grad` context. It will have a `grad_fn` function attached. This callback is used to backpropagate onto the computation graph to compute the gradients of preceding tensor nodes.

- **`torch.autograd.grad`** :

Here we provide `torch.ones_like(y)` as the `grad_outputs`.

```
>>> torch.autograd.grad(y, (x1, x2), torch.ones_like(y))
(tensor(0.7879), tensor(5.))
```

The above output is a *tuple* containing the two partial derivatives w.r.t. to the provided inputs respectively in order of appearance, i.e. `dL/dx1` and `dL/dx2`.

This corresponds to the following computation:

```
# dL/dx1 = dL/dy * dy/dx1 = grad_outputs @ 2*x1
# dL/dx2 = dL/dy * dy/dx2 = grad_outputs @ 5
```

- **`torch.autograd.backward`** : in contrast, it will mutate the provided tensors by updating the `grad` of the tensors that have been used to compute the output tensor and that require gradients. It is equivalent to the `torch.Tensor.backward` API. Here, we go through the same example by defining `x1`, `x2`, and `y` again. We call `backward` :

```
>>> # y.backward(torch.ones_like(y))
>>> torch.autograd.backward(y, torch.ones_like(y))
None
```

Then you can retrieve the gradients on `x1.grad` and `x2.grad` :

```
>>> x1.grad, x2.grad
(tensor(0.7879), tensor(5.))
```

In conclusion: both perform the same operation. They are two different interfaces to interact with the `autograd` library and perform gradient computations. The latter, `torch.autograd.backward` (equivalent to `torch.Tensor.backward`), is generally used in neural networks training loops to compute the partial derivative of the loss w.r.t each one of the model's parameters.

You can read more about how `torch.autograd.grad` works by reading through this other answer I made on: [Meaning of grad outputs in PyTorch's torch.autograd.grad](#).

Share Edit Follow Flag

edited Mar 23, 2024 at 21:34

answered Sep 12, 2021 at 12:50



Ivan

40.7k ● 8 ● 72 ● 117

2 Thank you very much, couldn't find such profound explanation anywhere.. – [unstableEquilibrium](#) Sep 18, 2021 at 2:22

There is no such parameter `grad_tensors` in [autograd.grad](#) – Kuo Mar 23, 2024 at 17:10

In addition to Ivan's answer, having `torch.autograd.grad` not accumulating gradients into `.grad` can avoid racing conditions in multi-thread scenarios.

3 Quoting PyTorch doc <https://pytorch.org/docs/stable/notes/autograd.html#non-determinism>

If you are calling backward() on multiple thread concurrently but with shared inputs (i.e. Hogwild CPU training). Since parameters are automatically shared across threads, gradient accumulation might become non-deterministic on backward calls across threads, because two backward calls might access and try to accumulate the same .grad attribute. This is technically not safe, and it might result in racing condition and the result might be invalid to use.

But this is expected pattern if you are using the multithreading approach to drive the whole training process but using shared parameters, user who use multithreading should have the threading model in mind and should expect this to happen. User could use the functional API torch.autograd.grad() to calculate the gradients instead of backward() to avoid non-determinism.

implementation details

https://github.com/pytorch/pytorch/blob/7e3a694b23b383e38f5e39ef960ba8f374d22404/torch/csrc/autograd/functions/accumulate_grad.h

Share Edit Follow Flag

answered Jan 30, 2022 at 22:41



[dontloo](#)

10.9k ● 4 ● 34 ● 54

This post is hidden. It was [deleted](#) 3 years ago by the post author.

`loss.backward()` does the backpropagation and accumulates the gradient of all the trainable parameters of your network. You need to call `optimizer.step()` to update the parameters' value. Don't forget to call `optimizer.zero_grad()` to empty out the accumulated gradients.

Share Edit Follow **Undelete** Flag

answered Sep 12, 2021 at 5:15



[Buoy Rina](#)

568 ● 1 ● 4 ● 9

Comments disabled on deleted / locked posts / reviews